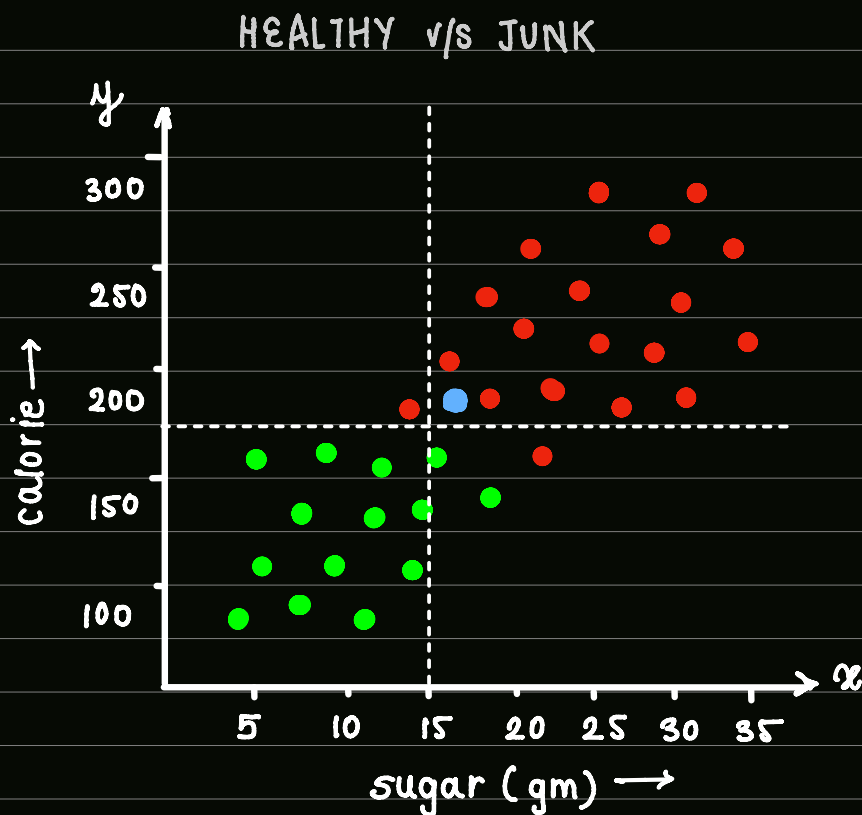


MACHINE LEARNING : DECISION TREES

~ Lekanksha A. Jain

GEOMETRIC intuition



low sugar + low calorie
= HEALTHY

high sugar + high calorie
= JUNK

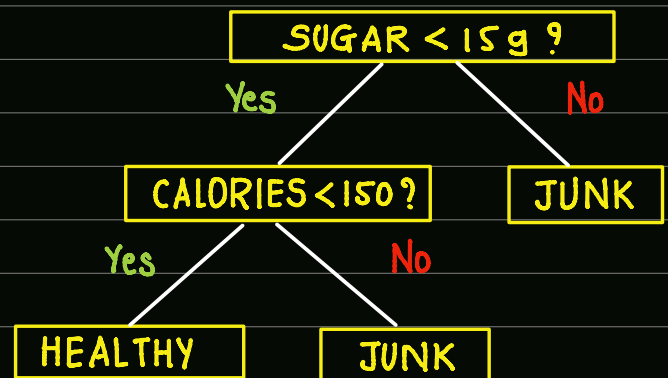
• Blue point (18g sugar and 210 calorie) which category does this data point fall into?

THIS IS WHERE A
DECISION TREE
COMES IN WORK!

- Decision Trees are nothing, but nested if else conditions :

- In simpler terms, it is like a flowchart :

```
if sugar < 15:  
    if calories < 150:  
        print ("Healthy")  
    else:  
        print ("Junk")  
else:  
    print ("Junk")
```



Definition : Decision Tree is a supervised machine learning algorithm that can be used for both :

- (a) Classification
- (b) Regression

→ model trained using data that already has correct answers (LABELS).

sorts and predicts category

predicts numerical value

HOW DOES DECISION TREE WORK?

eg. Golf Play Dataset:

independent features (I/P)

Target (O/P)

Day	Outlook	Temperature	Humidity	Wind	Play Golf
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Using a decision tree tell whether a person is going to play golf or not?

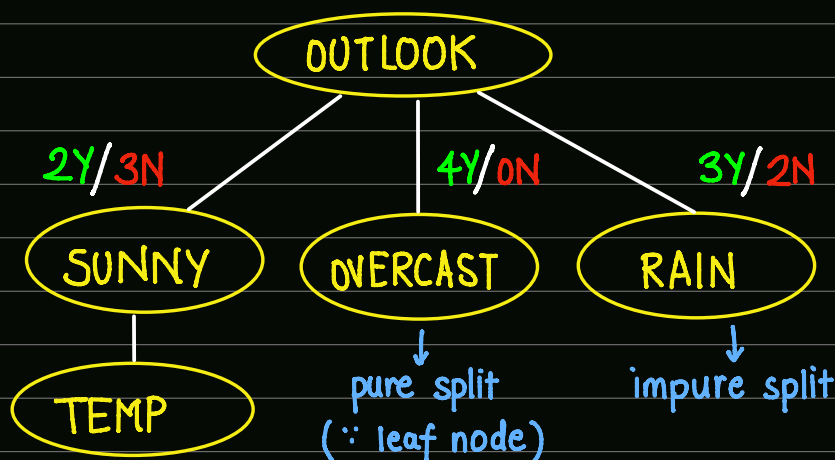
i. Count the total no. of Yes & No for playing golf

⇒ 9Y/5N

ii. Let's split based on Outlook feature:

- Unique categories in outlook?
SUNNY, RAIN and OVERCAST

Two questions arise here —



i. Mathematically, how do computers recognize a pure split?

A. using (a) Entropy
(b) Gini impurity

ii. Which feature to select for best split?

A. using Information Gain

split until a leaf node is found!

2} MATHEMATICS

[A] Entropy: used in Decision Tree to measure how impure/uncertain a dataset is; higher the entropy $\rightarrow$ more the impurity.

eg. If a dataset has equal number of Y and N (3Y/3N) then predicting the outcome is highly uncertain.
But if we have 3Y/0N, meaning, that there is no uncertainty left in predicting the outcome.

Mathematically, FORMULA:

i. For a dataset with n classes:

$$\text{Entropy}(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

where; S = dataset

p_i = probability of class i

$\log_2$ = logarithm to base 2

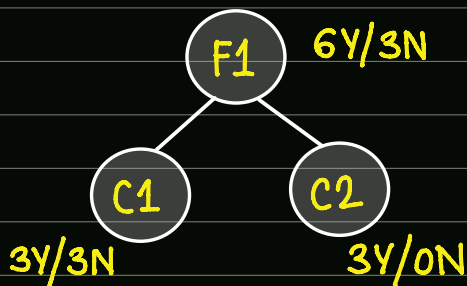
ii. For Binary Classification:

$$\text{Entropy}(S) = -p_Y \log_2 p_Y - p_N \log_2 p_N$$

p_Y = probability of YES

p_N = probability of NO

eg. Consider the following tree with feature F1 and category C1 & C2:



For C1: $p_Y = 3$, $p_N = 3$, total sample = 6
substituting in formula:

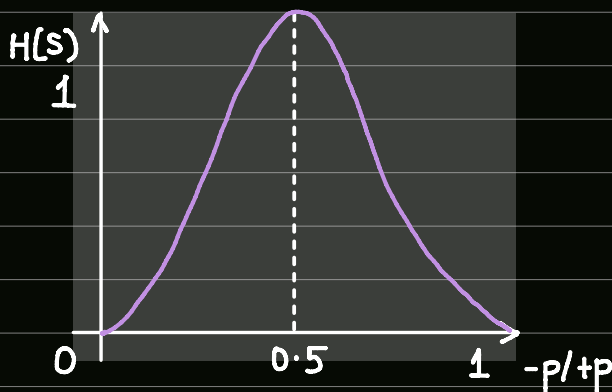
$$\text{entropy}(C1) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6}$$

$$\text{entropy}(C1) = 1 \text{ (highly impure)}$$

For C2: $p_Y = 3$, $p_N = 0$, total sample = 3
substituting in formula:

$$\text{entropy}(C2) = -\frac{3}{3} \log_2 \frac{3}{3} - \frac{0}{3} \log_2 \frac{0}{3}$$

$$\text{entropy}(C2) = 0 \text{ (pure split)}$$



$\therefore$ entropy ranges b/w 0 to 1 $\begin{cases} \rightarrow 0 \text{ (pure node)} \\ \rightarrow 1 \text{ (maximum impurity 50-50 split)} \end{cases}$

[B] Gini Index: measures how mixed the classes are in a dataset, it is used by CART algorithm to decide the best split.

Mathematically, FORMULA:

i. For a dataset with n classes:

$$\text{Gini}(S) = 1 - \sum_{i=1}^n p_i^2$$

where; S = dataset

p_i = probability of class i

ii. For binary classification:

$$\text{Gini}(S) = 1 - (p_Y^2 + p_N^2)$$

eg. Suppose a dataset has: healthy = 6, junk = 4, total = 10

$$\rightarrow \text{probability}_Y (\text{Healthy}) = \frac{6}{10} = 0.6$$

$$\rightarrow \text{probability}_N (\text{Junk}) = \frac{4}{10} = 0.4$$

$$\therefore \text{Gini} = 1 - ((0.6)^2 + (0.4)^2)$$

$$\therefore \text{Gini} = 1 - 0.52$$

$$\therefore \text{Gini} = 0.48$$

Gini values	meaning
0	pure node
0-0.5	increasing impurity
0.5 binary case	maximum impurity 50-50

[C] Information Gain: tells us how useful a feature is for splitting data into groups; the feature with highest information gain means a better split.

Mathematically, FORMULA:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{val}} \frac{|S_v|}{|S|} H(S_v)$$

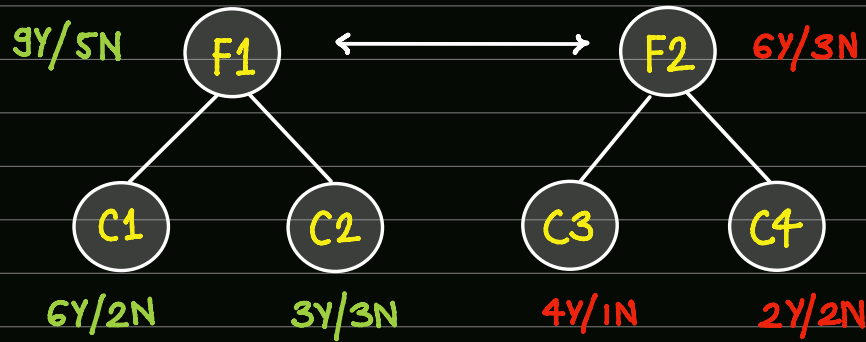
entropy
parent node

weighted entropy
child node

; S_v = sample in
child node

S = sample in
root node

eg. Consider the given two subtrees, calculate Information Gain to identify which split to choose:



For F1:

i. Entropy $H(S)$ of root node:

$$= -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14}$$

$$= 0.94$$

ii. Entropy C1:

$$= -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8}$$

$$= 0.81$$

iii. Entropy C2:

since the split is 50-50
 $\therefore$ entropy of C2 = 1.

$\Rightarrow$ Gain $(S, F1)$

$$= 0.94 - \left[\frac{8}{14} \times 0.81 + \frac{6}{14} \times 1 \right]$$

$$= 0.049$$

For F2:

i. Entropy $H(S)$ of root node:

$$= -\frac{6}{9} \log_2 \frac{6}{9} - \frac{3}{9} \log_2 \frac{3}{9}$$

$$= 0.91$$

ii. Entropy C3:

$$= -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5}$$

$$= 0.72$$

ii. Entropy C4:

since the split is 50-50
 $\therefore$ entropy of C4 = 1

$\Rightarrow$ Gain $(S, F2)$

$$= 0.91 - \left[\frac{5}{9} \times 0.72 + \frac{4}{9} \times 1 \right]$$

$$= 0.073$$

• Conclusion: since, $0.073 > 0.049$

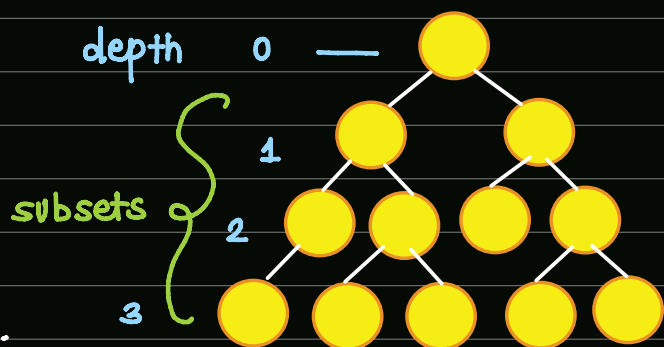
$\therefore$ We choose F2 as a better feature for splitting.

3) mechanism

1. Recursive partitioning: the process of repeatedly dividing the dataset into smaller and smaller subsets based on the best feature until a stopping condition is met.

⇒ STEPS:

- i. start with complete dataset.
- ii. find best feature using Entropy / Gini.
- iii. split data into subsets.
- iv. repeat same process for each subset.
- v. continue until stopping criteria is reached.



2. Stopping Criteria: a decision tree stops growing when one of the following conditions is met —

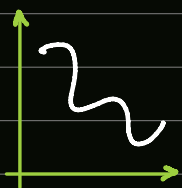
- (a) Pure node: all samples belong to same class (no further split is needed).
- (b) Maximum depth reached: the predefined maximum depth (max_depth) is reached.
- (c) Minimum samples to split: number of samples is too small to create another split ($\text{min_samples_split} = 10$), suppose 8 samples then can't split.
- (d) No Information Gain: further splitting does not improve the model ($I \cdot G = 0$).

3. Concept of overfit and underfit :-

OVERFIT

- model that learns training data too well including noise, so it performs poorly on new data.

- very complex tree
- high training accuracy
- low testing accuracy

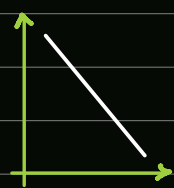


eg. A teacher predicts student's result based on marks, attendance, study hours, assignments, project, library visits, etc.

UNDERFIT

- model that is too simple and fails to capture important patterns in the data.

- very simple tree
- low training accuracy
- low testing accuracy



eg. A teacher predicts student's result only based on attendance & ignores other features like marks, assignments, etc.

HOW CAN WE MAKE A DECISION TREE SIMPLER WITHOUT LOSING MUCH ACCURACY ?

Ans. By Pruning: Pruning is the process of removing unnecessary branches from a decision tree to reduce overfitting and generalization.

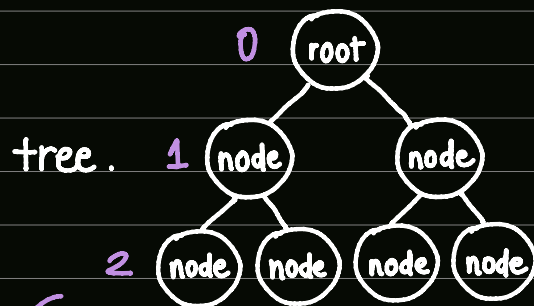
- Types of pruning: $\begin{cases} \rightarrow (a) \text{ PRE-PRUNING} \\ \rightarrow (b) \text{ POST-PRUNING} \end{cases}$

➤ PRE-PRUNING (Early stopping) :-

- technique in which the growth of the decision tree is stopped early based on predefined conditions, preventing the tree from becoming too complex.

A> Maximum Depth :

- meaning : limits the maximum depth of the tree.
- syntax : `DecisionTreeClassifier (max_depth = 2)`
- example : `max_depth = 2`



the tree cannot grow beyond 2 levels.

B> Minimum Samples to split :

- meaning : a node must have at least this many samples to split.
- syntax : `DecisionTreeClassifier (min_samples_split = 5)`
- example : `min_samples_split = 5`

node 8 samples
✓ can split

node 3 samples
✗ cannot split

C> Minimum Samples per Leaf :

- meaning : every leaf node must contain at least the specified number of samples.
- syntax : `DecisionTreeClassifier (min_samples_leaf = 3)`
- example : `min_samples_leaf = 3`

Before split : 10 samples After split : left = 8 right = 2 invalid ✗
OR left = 5 right = 5 valid ✓

D> Minimum Information Gain / Minimum Impurity Decrease:

- meaning : a split is performed only if it reduces impurity by at least a specified amount.
- syntax : `DecisionTreeClassifier (min_impurity_decrease = 0.05)`
- example : required minimum = 0.05

Split A:
Information Gain = 0.12
SPLIT ✓

Split B:
Information Gain = 0.02
STOP ✗

2> POST-PRUNING :-

- grows the complete tree first and then removes unnecessary branches after creation.

- syntax : `DecisionTreeClassifier (ccp_alpha = 0.02)`

ccp_alpha = cost complexity pruning

- example : Suppose: BEFORE PRUNING

accuracy = 100%

leaf nodes = 15

AFTER PRUNING

accuracy = 100%

leaf nodes = 8

When to use pre & post ?

Pre-pruning when the dataset is very large and Post-pruning when accuracy is more important.

In practice, post-pruning is generally preferred because it first learns the complete structure of the data and then removes only the unnecessary branches, resulting in a simpler model with good predictive performance.

4> CODING - implementation

STEP 1: Importing Libraries :

- required libraries are :

(a) Pandas → data manipulation

(b) matplotlib → visualization

(c) seaborn → visualization (scatter plot)

(d) scikit learn

↳ `sklearn.datasets import dataset_name`

↳ `sklearn.model_selection import train_test_split`

↳ `sklearn.tree import DecisionTreeClassifier`

↳ `sklearn.metrics import accuracy_score, confusion_matrix`

STEP 2: Load Dataset

STEP 3: Data and Target :

$x = \text{data}$ → INPUT features

$y = \text{target}$ → OUTPUT labels

STEP 4: Create dataframe :

convert database into tabular form

STEP 5: Understand the dataset :

`df.shape` → rows and columns

`df.describe()` → statistical summary (min, max, mean, median)

`df.info()` → datatypes and missing values

STEP 6: Visualise dataset

STEP 7: Train-test split :

data is split into training 80% and testing 20%

STEP 8: Create the model :

this includes hyperparameters like:

• `criterion` — splitting criteria (Gini/Entropy)

• `random_state` — for reproducible results

• `max_depth` — specify max depth of tree

• `ccp_alpha` — post pruning parameter

STEP 9: Train the model using training data :

• **SYNTAX:** `model.fit(X_train, y_train)`

STEP 10: Make predictions :

predict labels for test data

STEP 11: Accuracy :

• Formula : $\text{acc} = \frac{\text{correct predictions}}{\text{total predictions}}$

STEP 12: Confusion Matrix :

compare actual v/s predicted values

STEP 13: Decision Tree Visualization

visualize the trained tree

STEP 14: Feature Importance :

identify the most influential features.

⇒ KEY TAKEWAYS :-

- Decision Trees are supervised learning algorithms used for classification and Regression.
- They make predictions using a series of if-else rules.
- The best split is chosen using Entropy, Gini Index, Information Gain.
- Trees are built using Recursive Partitioning.
- Growth stops based on stopping criteria.
- Overfitting can be reduced using pre and post pruning.
- The model is evaluated using accuracy and confusion matrix.
- Feature importance

— X —

LinkedIn : Lekanksha Jain

Github : lekhja